Frontend Build Guidelines: UI Only

Prepared by:
Michael Hall
Sprint Digital Pty Ltd
m.hall@sprintdigital.com.au
1300 359 892

Date: 18 May 2026
Version: 0.1 DRAFT

Overview

These guidelines are for building a UI layer **only** in React Native Expo. Sprint Digital will handle all backend API work, data wiring, authentication, and state management.

Tech Stack

Area	Library	Why
Framework	**React Native via Expo**	Our deployment target
Styling & theming	**Tamagui**	All our components are built on it - no StyleSheet, no NativeWind, no styled-components
Navigation / routing	**Expo Router (file-based)**	We use file-based routing with route groups
Forms	**react-hook-form**	Our form components are wrappers around its `Controller` - they won't work with Formik or uncontrolled inputs
Icons	**lucide-react-native**	Our `Icon` component wraps it
Validation	**yup**	For form schema validation alongside react-hook-form

Do NOT introduce Tailwind/NativeWind, styled-components, React Navigation (stack/tab navigators), Formik, or any other styling or form library.

Use Our Components: Do Not Rebuild

Every screen should be composed from our `Indi`-prefixed components. Here is what's available, use them instead of building your own:

Layout: `IndiView`, `IndiXStack` (horizontal), `IndiYStack` (vertical), `IndiCard`, `IndiOutlineCard`, `IndiSeparator`, `Container`

Text: `IndiText`, `IndiH1`, `IndiH2`, `IndiH3`, `IndiH4`, `IndiLabel`, `IndiParagraph` - never use raw `<Text>` from React Native

Buttons: `IndiButton` - uses `type` (solid/outline/ghost/link) + `color` (primary/secondary/red) + `size` (xs/sm/md/lg). Do not create custom button components.

Inputs: `IndiInput`, `IndiLabelInput`, `IndiCurrencyInput`, `IndiPhoneInput`, `IndiEmailInput`, `IndiSearchInput`

Form Controls (react-hook-form wrappers): `FormTextInput`, `FormNumberInput`, `FormSelect`, `FormCheckbox`, `FormAbnInput`, `FormAddress`, `FormAutocomplete`

Selects: `IndiSelect` (auto-upgrades to autocomplete when >10 items), `IndiAutoComplete`

Domain Selects (pre-wired to lookup data): `StateSelect`, `RoleSelect`, `TeamSelect`, `DepartmentSelect`, `ServiceTypeSelect`, `LicenseRoleSelect`

Tables: `IndiTable<T>` - generic, sortable, paginated. Define columns with `IndiColumn<T>[]`.

Overlays: `IndiModal`, `IndiDrawer`, `IndiDialog`, `IndiAlertDialog`, `Toast`

Navigation: `Tabs`, `NavigationTile`, `Breadcrumb`

Misc: `IndiBadge`, `IndiTag`, `AvatarIcon`, `IndiTooltip`, `IndiPagination`, `IndiLoader`, `IndiStepper`, `IndiSwitch`, `IndiCheckbox`, `IndiRadio`, `SignaturePad`, `DatePicker`

Styling Rules

Use Tamagui tokens, never hardcoded values

```
None
// ✅ Correct
<IndiText color="$textPrimary">Hello</IndiText>
<IndiView backgroundColor="$containerBg" borderRadius="$default" padding="$4">
```

```
// ❌ Wrong - hardcoded colours and values
<IndiText color="#333333">Hello</IndiText>
<IndiView style={{ backgroundColor: 'white', borderRadius: 8, padding: 16 }}>
```

Key token categories

- **Text:** `$textPrimary`, `$textSecondary`, `$textNeutral`, `$textPlaceholder`, `$textDisabled`
- **Backgrounds:** `$containerBg`, `$pageBg`, `$modalBg`
- **Borders:** `$border`, `$inputBorderDefault`, `$inputBorderError`
- **Buttons:** `$buttonSolidPrimaryBg`, `$buttonOutlineRedBorder`, etc.
- **Semantic colours:** `$Green500`, `$Red500`, `$Blue500`, `$Orange500`, `$Purple500`, `$Neutral200`
- **Spacing:** `$1` through `$10`
- **Fonts:** `$body`, `$heading`

Responsive breakpoints

Use Tamagui media queries for responsive layouts:

```
None
<IndiView width="100%" $gtMd={{ width: '50%' }} $gtLg={{ width: '33%' }}>
```

Available: `$xs`, `$md`, `$gtMd`, `$gtLg`

Screen Structure & File Organisation

File-based routing with Expo Router

```
None
app/
├── (app)/
│   ├── (business)/          # Business user screens
│   │   ├── dashboard.tsx
│   │   ├── jobs/
│   │   │   ├── index.tsx    # Job list
│   │   │   └── [id].tsx     # Job detail
```

```
|   |   └── settings.tsx
|   ├── (candidate)/         # Candidate user screens
|   |   ├── dashboard.tsx
|   |   ├── applications/
|   |   |   └── index.tsx
|   |   └── profile.tsx
|   └── _layout.tsx
├── (auth)/
|   ├── login.tsx
|   └── register.tsx
└── _layout.tsx
```

Every entity/view must exist for BOTH user types

If business users have a "Jobs" list, candidates must have a corresponding view (e.g. "Available Jobs" or "My Applications"). Design both sides of every feature.

Screen template

None

```
import { Container } from '@/components';
import { IndiH1, IndiText, IndiYStack } from '@/components';

export default function JobListScreen() {
  return (
    <Container>
      <IndiYStack gap="$4" padding="$4">
        <IndiH1>Jobs</IndiH1>
        {/* Screen content using Indi components */}
      </IndiYStack>
    </Container>
  );
}
```

How To Handle Data in Screens

Since you're only building the UI, every screen needs **mock data** so we can see exactly what the design expects.

Use static mock data at the top of each screen file

```
None
// -- Mock Data (Sprint Digital will replace with API calls) --
const MOCK_JOBS = [
  {
    id: '1',
    title: 'Senior Plumber',
    location: 'Sydney, NSW',
    status: 'open',
    postedAt: '2026-05-10',
    applicantCount: 12,
  },
  {
    id: '2',
    title: 'Electrician Level 2',
    location: 'Melbourne, VIC',
    status: 'closed',
    postedAt: '2026-04-28',
    applicantCount: 34,
  },
];
```

Rules for mock data

1. **Use realistic Australian data** - real suburb names, valid-looking ABNs, Australian phone format (+61), AUD currency
2. **Use TypeScript interfaces** for every mock data shape - this tells us exactly what fields each screen expects:

```
None
interface Job {
  id: string;
  title: string;
  location: string;
  status: 'open' | 'closed' | 'draft';
```

```
  postedAt: string;      // ISO date
  applicantCount: number;
}
```

3. **Include edge cases** - show at least one example with: empty/null optional fields, long text that might wrap, zero counts, error states
4. **Mock enough rows** for lists/tables - at least 5-8 items so we can see pagination and scroll behaviour
5. **Keep all mock data in the same file** as the screen that uses it, clearly labelled

What We Need From Every Screen

Each screen you deliver should include:

Both user type variants

Every feature must have screens for both **Business** and **Candidate** users. If a screen only makes sense for one type, document why.

All interactive states

For every form or interactive element, build out:

- **Empty state** - what does it look like with no data?
- **Populated state** - normal usage with data
- **Loading state** - use `IndiLoader` or `LoadingContainer` as a placeholder
- **Error state** - show form validation errors using the `error` prop on form components
- **Success state** - what happens after a successful action? (Toast? Redirect? Confirmation?)

Typed mock data interfaces

As described above - TypeScript interfaces for every data shape used in the screen.

Navigation wired up

Use Expo Router's `Link` and `router.navigate()` to connect screens together. We need to see the full user flow, not isolated pages.

Responsive consideration

Screens should work for the web first but mobile responsiveness sounds like it is required, this will also assist with native app builds in future. Also consider tablet widths using Tamagui media queries (`$gtMd`, `$gtLg`).

Naming Conventions

Thing	Convention	Example
Screen files	`camelCase.tsx`	`jobList.tsx`, `candidateProfile.tsx`
Components (if you must create a screen-level one)	`PascalCase` with no prefix	`JobCard`, `ApplicationRow` - **not** `IndiJobCard` (the `Indi` prefix is reserved for our library)
Mock data constants	`UPPER_SNAKE_CASE`	`MOCK_JOBS`, `MOCK_CANDIDATES`
TypeScript interfaces	`PascalCase`	`Job`, `Candidate`, `Application`
Route params	`camelCase`	`jobId`, `applicationId`

Form Patterns

Always use react-hook-form with our form wrapper components:

```
None
import { useForm } from 'react-hook-form';
import { FormTextInput, FormSelect, FormCheckbox } from '@/components/forms';
import { IndiButton } from '@/components/buttons';
import * as yup from 'yup';
import { yupResolver } from '@hookform/resolvers/yup';
```

```
interface CreateJobForm {
  title: string;
  description: string;
  location: string;
  serviceType: string;
  isUrgent: boolean;
}

const schema = yup.object({
  title: yup.string().required('Job title is required'),
  description: yup.string().required('Description is required'),
  location: yup.string().required('Location is required'),
  serviceType: yup.string().required('Select a service type'),
  isUrgent: yup.boolean().default(false),
});

export default function CreateJobScreen() {
  const { control, handleSubmit } = useForm<CreateJobForm>({
    resolver: yupResolver(schema),
    defaultValues: { title: '', description: '', location: '', serviceType: '',
isUrgent: false },
  });

  const onSubmit = (data: CreateJobForm) => {
    // Sprint Digital will replace with API mutation
    console.log('Form submitted:', data);
    Toast.success({ message: 'Job created successfully' });
  };

  return (
    <Container>
      <IndiYStack gap="$4" padding="$4">
        <IndiH1>Create Job</IndiH1>
        <FormTextInput control={control} name="title" label="Job Title" />
        <FormTextInput control={control} name="description" label="Description" />
        <FormTextInput control={control} name="location" label="Location" />
        <FormSelect control={control} name="serviceType" label="Service Type"
data={MOCK_SERVICE_TYPES} />
        <FormCheckbox control={control} name="isUrgent" label="Mark as urgent" />
        <IndiButton type="solid" color="primary"
handlePress={handleSubmit(onSubmit)}>
          Create Job
        </IndiButton>
      </IndiYStack>
    </Container>
  );
```

```
}
```